System Verification and Validation Plan for Software Engineering

Team 22, Ghoti
Hemal Rajput
Gurmanjot Minhas
James Araiche
Lily Porter
Sarah Dorfman

November 3, 2026

Revision History

Date	Version	Notes
November 2nd, 2026	1.0	Initial Revision of VnV Plan

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 1
- 2.4 Relevant Documentation . . . 2

3 Plan **2**
- 3.1 Verification and Validation Team . . . 2
- 3.2 SRS Verification . . . 3
- 3.3 Design Verification . . . 4
- 3.4 Verification and Validation Plan Verification . . . 5
- 3.5 Implementation Verification . . . 5
- 3.6 Automated Testing and Verification Tools . . . 6
- 3.7 Software Validation . . . 6

4 System Tests **7**
- 4.1 Tests for Functional Requirements . . . 7
  - 4.1.1 Audio capture . . . 7
  - 4.1.2 Session Management . . . 9
  - 4.1.3 Speech Analysis . . . 12
  - 4.1.4 Results and Data Visualization . . . 13
  - 4.1.5 Progress and Data Management . . . 15
  - 4.1.6 Practice Activities . . . 18
  - 4.1.7 Reliability and Offline Behavior . . . 20
- 4.2 Tests for Nonfunctional Requirements . . . 21
  - 4.2.1 Accuracy and responsiveness . . . 21
  - 4.2.2 Usability and patient protection . . . 22
  - 4.2.3 Maintainability and portability . . . 24
  - 4.2.4 Reliability, security and release review . . . 25
  - 4.2.5 Legal and Licensing . . . 27
- 4.3 Traceability Between Test Cases and Requirements . . . 28

5 Unit Test Description **29**
5.1 Unit Testing Scope . . . 30
5.2 Tests for Functional Requirements . . . 30
5.2.1 Module 1 . . . 30
5.2.2 Module 2 . . . 31
5.3 Tests for Nonfunctional Requirements . . . 31
5.3.1 Module ? . . . 31
5.3.2 Module ? . . . 32
5.4 Traceability Between Test Cases and Modules . . . 32

6 Appendix **33**
6.1 Symbolic Parameters . . . 33

7 Appendix — Generative AI Use Disclosure **34**
7.1 AI Tools Used . . . 34
7.2 How and Where Used . . . 34
7.3 Verification of AI-Generated Content . . . 35

8 Appendix — Reflection **37**

List of Tables

1 Developer and Tester Assignments . . . 3
2 Traceability Between System Tests and Requirements . . . 28
3 List of AI Tools Used . . . 34
4 Usage Details for All AI Tools Used . . . 35
[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

[symbols, abbreviations, or acronyms — you can simply reference the SRS (Author, 2019) tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

2.1 Summary

[Say what software is being tested. Give its name and a brief overview of its general functions. —SS]

2.2 Objectives

[State what is intended to be accomplished. The objective will be around the qualities that are most important for your project. You might have something like: "build confidence in the software correctness," "demonstrate adequate usability." etc. You won't list all of the qualities, just those that are most important. —SS]

[You should also list the objectives that are out of scope. You don't have the resources to do everything, so what will you be leaving out. For instance, if you are not going to verify the quality of usability, state this. It is also worthwhile to justify why the objectives are left out. —SS]

[The objectives are important because they highlight that you are aware of limitations in your resources for verification and validation. You can't do everything, so what are you going to prioritize? As an example, if your system depends on an external library, you can explicitly state that you will assume that external library has already been verified by its implementation team. —SS]

2.3 Custom Documents

[Summarize the CDs (Custom Documents) that were tackled by this project. CDs can include usability testing, code walkthroughs, user documentation, formal proof, GenderMag personas, Design Thinking, etc. CDs should have already been approved by the course instructor as included in your problem statement. You can use a pull request to update your CDs (in TeamComposition.csv or Repos.csv) if your plan changes as a result of the VnV planning exercise. —SS]

2.4 Relevant Documentation

[Reference relevant documentation. This will definitely include your SRS and your other project documents (design documents, like MG, MIS, etc). You can include these even before they are written, since by the time the project is done, they will be written. You can create BibTeX entries for your documents and within those entries include a hyperlink to the documents. —SS]

Author (2019)

[Don't just list the other documents. You should explain why they are relevant and how they relate to your VnV efforts. —SS]

3 Plan

The following section outlines a high-level verification and validation plan for PronouncedFish. Section 3.1 covers the verification and validation team, and a general description of each person's role. Section 3.2 and 3.3 go into the techniques used for SRS and Design verification (respectively). Section 3.4 details the steps used to verify the verification and validation plan itself. Section 3.5 deals with the eventual verification methods anticipated during the implementation stages, while section 3.6 talks about some automated testing frameworks and verification tools that will be used. All of these sections delve into software verification methods: how do we know we are building the right product. Section 3.7 wraps up by going into some system validation methods: how do we know we are building the product right.

3.1 Verification and Validation Team

The Verification and Validation Team will be composed of all team members and supervisors. In general, the software team will be in charge of initial stages of testing, while the supervisors will be brought in near latter stages to reinforce the student validation. Some steps will seek feedback from the assigned peer review group as well.

The software team will conduct preliminary SRS verification, and make sure the functionality matches what is outlined in the SRS. Verified elements will then be passed forward to the supervisors to ensure they truly match what the users need. It is important to note that since the supervisors work very closely with SLPs, requirements were heavily solicited through them.

Therefore, this process cross verifies whether the perceived implementation of those requirements are in line with what is needed.

Furthermore, the software team will be in charge of unit tests, integration tests, system tests, and regression tests for all software. These should be added to the Github CI/CD so that they are checked upon any code changes. These tests should be curated by someone besides the primary developer of those features.

Any system design decisions will be verified on a rolling basis by another team member. They should be done frequently so as to not hold back development. Mutation testing should also be completed by a different group member to ensure developer bias does not interfere with testing.

The following table outlines the assigned tester for each developer:

Developer	**Tester**
Hemal Rajput	Gurmanjot Minhas
Gurmanjot Minhas	James Araiche
James Araiche	Lily Porter
Lily Porter	Sarah Dorfman
Sarah Dorfman	Hemal Rajput

Table 1: Developer and Tester Assignments

3.2 SRS Verification

The SRS verification will be done by the project supervisors. The supervisors are interested in using a version of this project for their own teaching, and act as the liaison between the developers and the SLPs. Therefore, they are the ideal representative to validate what is being done is right.

SRS verification will be handled using an agile framework, and touched on during each biweekly meeting. Starting from the core of the system, more features will be fleshed out between each meeting. Prior to development beginning, all related requirements will be touched on and refined with the supervisors to ensure they align with what they had in mind. Following development, a short demonstration will compile the implementations of all those requirements so they may be revalidated. Since each requirement is attributed to one main goal, relevant requirements may be easily split up

based on the current focus. This timeline is not concrete and may vary based on the nature of the implemented features and importance of other matters.

The requirement validation portion will be more of an interview process where the supervisors read each requirement and comment on whether it makes sense. The eventual demonstration will start with the software team showing an overview of what has been put in. This will be followed by time for the supervisors to test their own cases and give additional feedback. This will allow for the SRS document or implementation to be revised as needed. The GitHub issue tracker will be used to create issues when needed.

3.3 Design Verification

Design verification will be conducted by a mixture of the group members, reviewers, and supervisors.

The supervisors will be consulted for user interface verification. For each main page and visualization, the supervisors will be shown a Figma (high-fidelity wireframe drawing) to review. Based on this, they will determine if the interface is too complicated, needs more explanation, or has any usability concerns. Since they have direct interaction with SLPs, they have a good idea of their needs, abilities, and day to day, making them the best group to validate this.

Additionally, the supervisors will have to verify all speech processing elements since the software team are not linguistics experts. The plan is for this to be addressed during the scheduled biweekly meetings. Specifically, all the phoneme representations, comparison metrics, and conclusions will need to be verified for relevance and correctness.

On a grander scheme, the software architecture will go through several iterations of peer review. The software team will break off into several smaller groups and create initial drawings for some interface. The drawings will then be rotated to another sub group and refined. Once the entire team is satisfied with the drawings, they will be presented to a peer review group for further evaluation. They will be looking at these designs with a fresh perspective, so they will likely notice any gaps that were missed by the development team.

There will also be a checklist created based on the different user interface requirements, functional requirements, and likely changes sections of the SRS. These shall accompany any peer reviews and supervisor reviews as a complementary tool.

3.4 Verification and Validation Plan Verification

The Validation and Verification plan will be verified by a mixture of formal and informal methods. For the SRS VnV plan, the team members will verify that each requirement from the SRS has a traceable verification or validation method assigned to it. All software classes must have a corresponding test suite for unit, integration, system, and regression testing as well. To ensure that this test suite is complete, it must achieve complete test coverage (see 3.6 for a definition of complete), and successfully report failures upon mutation testing. Mutation testing will be conducted by the assigned tester from 3.1. Clear acceptance criteria and metrics for non-functional requirements must be present, and approved by the supervisors. Once this is all done, the peer review team will verify the VnV plan using a checklist outlining the noted elements in detail.

3.5 Implementation Verification

The implementation will be verified by using tests of varying granularity. At the lowest level, unit tests shall be introduced to test each component in isolation. These tests must cover all branches of execution to make sure the unit is fully covered by using mocks and stubs to abstract out dependencies on other classes. Moving up one level, there must be integration tests to connect each element of the code to make sure they work with each other correctly. For example, if some class is written to take measurements in the imperial system and another in the metric system, then individually their expected outputs will be correct. However, when working with each other, the components will fail. At the highest level, there should be system tests to test the main modules as a whole. The assigned tester will be responsible for writing these tests for their corresponding developer from 3.1.

Beyond the code level, the implementation must also match what is desired by the stakeholders. This overlaps with design verification, but will be done by demonstrating the system to the supervisors and letting them play around with it. This will help them catch any usability mistakes or quality of life enhancements that they expected, but were not reflected by the implementation. Static analysis tools will also be useful to ensure all coding standards and conventions are met. Ruff and Prettier will be used to lint the corresponding Python / JavaScript code. The team will also perform code reviews during the integration stages. This lets the developers of each

module iron out any inconsistencies between their code and to ensure it is of an acceptable standard, without making it too frequent.

All code based tests will be run with Github continuous integration to ensure that no component interactions are broken when pushing new code to the main branch.

3.6 Automated Testing and Verification Tools

The coding standards and linting are mentioned in the Development Plan document. To recall:

- Python: PEP 8, enforced with Ruff for Linting
- JavaScript: the Airbnb JavaScript Style Guide, enforced with Prettier

Unit testing and Integration testing will be done using the Pytest framework. This framework is picked due to its ability to support both unit and integration testing rather than one or the other. These tools will be continuously enforced using Github CI/CD on every pull request. Tests are expected to achieve complete coverage, along with coverage of relevant edge cases and erroneous cases. This will be measured by using the Pytest-Cov coverage frameworks, which provides a report of line, branch, and function coverage. The goal is to achieve 90-100% statement coverage, and 80% branch coverage in general. Critical components should achieve 100% branch coverage. Relevant cases will be verified beforehand during the implementation verification phase.

3.7 Software Validation

To validate that the product is being built right, it must first ensure that it is built for the correct user group. While the system architecture will support adding multiple SLP interfaces, the main focus is on child SLPs. Therefore, all validation will be done using openly available child audio data from TalkBank. This is a well reputable platform used by researchers to share data and has high quality annotated speech datasets. Their privacy requirements will be strictly followed. Some of the data may be used to train a speech recognition model, but some will be explicitly separated off for validation and test phases.

Next, it is important to ensure that SLPs actually want the visualizations and features provided. This can be validated by going over a mixed task based inspection of the SRS and corresponding implementation demonstration with the supervisors. For example, if visualization A is described in the SRS, the corresponding implementation will be shown and approved or rejected by the supervisor.

Every small matter mentioned within this section will not be explicitly checked with the supervisors. There is limited time with them, so the mentioned strategies will be used when appropriate and for key checkpoints.

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

The subsections below outline all necessary system tests associated with the functional requirements of PronouncedFish.

4.1.1 Audio capture

1. ST-F01 Capture audio

   Control: Manual

   Initial State: A patient session is active; microphone is configured; no speech attempts have been made.

   Input: An attempt is recorded for a target word.

   Output: One audio recording is captured and linked to the session, patient and only the corresponding attempt.

   Test Case Derivation: **FR-AC1** specifies the capture of a patient's spoken response for each attempt.

   How test will be performed: Record an attempt through the session UI, then open the attempt's saved record and play the audio back. Confirm the recording is linked to the correct session, patient, and attempt only.

2. ST-F02 Isolate attempts

   Control: Manual

   Initial State: A patient session is active; microphone is configured; no speech attempts have been made.

   Input: Two different single word recordings/attempts.

   Output: Each attempt has its own audio and corresponding analysis. Audio is used only for its own attempt.

   Test Case Derivation: Audio is analyzed based only on the corresponding attempt as specified in **FR-AC2**.

   How test will be performed: Record two different attempts through the session UI, then open each attempt's saved record. Confirm each attempt has its own audio and its own analysis, and that neither attempt's audio appears in the other's analysis.

3. ST-F03 Reject unusable recordings

   Control: Automatic

   Initial State: A patient session is active; thresholds exist for clean audio for noise, clipping, and duration.

   Input: An unclean audio recording/attempt. A clean audio recording/attempt.

   Output: Noisy, clips or duration failures in attempts are correctly flagged and rejected.

   Test Case Derivation: Unclean audio is correctly flagged, which can be signaled to the SLP, as specified in **FR-AC3**.

   How test will be performed: Record or submit audio that is noisy, clipped, or too short/long through the session UI, and confirm each is flagged to the SLP and not analyzed. Then record a clean attempt to confirm it is accepted.

4. ST-F04 Recording retry

   Control: Manual

Initial State: A patient session is active; microphone is configured.

Input: An unclean audio recording/attempt.

Output: System prompts that the input is flagged as unclean and rejected; the system prompts a retry option.

Test Case Derivation: **FR-HA1** requires the system to prompt the user when noisy (less than 15 dB) microphone inputs are passed in, and allow them to retry that input.

How test will be performed: Play the noisy sample through the microphone during an attempt. Confirm the attempt is not accepted. Retry the attempt with clean audio and confirm the retry succeeds.

4.1.2 Session Management

1. ST-F05 Configure a session

   Control: Manual

   Initial State: A patient session is active; microphone is configured; no speech attempts have been made.

   Input: Select a patient from a list, add corresponding target phonemes and items. Attempt to select a phoneme that is not on the predetermined list.

   Output: The session definition accurately contains the selected patient, the target phonemes and items. Selectable phonemes are limited to the predetermined list.

   Test Case Derivation: **FR-SM1** specifies that session setup requires patient selection and selection of target phonemes and items. Selectable phonemes are limited to those covered by the predetermined speech-challenge list, as specified in **FR-SA1**.

   How test will be performed: Use the system UI to select a patient, target phoneme, and target item. Attempt to select an invalid phoneme and confirm it is not selectable or is rejected. Inspect the created session record to verify the stored patient, phonemes, and items.

2. ST-F06 Attempt and session controls

Control: Manual

Initial State: A patient session is active.

Input: Attempts are started, repeated, skipped, and ended; the session is ended.

Output: Start begins an attempt; repeat repeats the current attempt; skip advances without recording a valid attempt; end attempt closes the attempt; end session closes the session and prevents further attempts.

Test Case Derivation: **FR-SM2** requires the system to allow the SLP to start, repeat, skip, and end each attempt, and to end the session.

How test will be performed: Execute each control in order through the session UI. After each action, inspect the UI state and backend attempt/session status to confirm the expected state change.

3. ST-F07 Record support level per attempt

   Control: Manual

   Initial State: A patient session is active with three attempts recorded.

   Input: The support level for each attempt is recorded as independent, semantic cue, and/or repetition.

   Output: Each attempt stores its selected support level and associates it with the correct attempt.

   Test Case Derivation: **FR-SM3** requires the system to record whether the target was produced independently, after a semantic cue, or after repetition.

   How test will be performed: Set the support level for each attempt through the UI. Query the attempt records and verify each stored support level matches the attempt it was entered for.

4. ST-F08 SLP override of classification

   Control: Manual

   Initial State: An attempt has been classified by the system to contain a speech error.

Input: The classification is overridden to be marked as "correct" instead.

Output: The attempt is stored and displayed as overridden.

Test Case Derivation: **FR-SM4** requires the system to allow the SLP to override the system's classification of any attempt. The system is meant to be assistive, not diagnostic.

How test will be performed: Use the override control in the UI. Inspect the stored attempt and the result view to confirm the SLP override is applied and the original system classification is still recorded.

5. ST-F09 Two-factor authentication and password recovery

   Control: Manual

   Initial State: An SLP account exists with two-factor authentication enrolled and a recovery contact configured.

   Input: Log in with the correct password but without the second factor; then provide a valid second factor. Initiate the password recovery flow.

   Output: Access is denied before the second factor is provided; access is granted after a valid second factor; the password recovery flow is available and completes successfully.

   Test Case Derivation: **FR-HA6** requires the system to have two-factor authentication and a password recovery system in place.

   How test will be performed: Attempt login without the second factor and confirm denial. Complete two-factor authentication and confirm access. Run the password recovery flow with the test account and confirm the password reset completes.

6. ST-F10 Patient consent code before recording

   Control: Manual

   Initial State: A patient session is started; a mock trusted device is available to receive a one-time code; recording is not yet authorized.

   Input: Attempt to authorize recording before entering the code; then enter the valid one-time code into the interface; start recording; attempt to reuse the same code.

Output: Recording is blocked until the valid one-time code is entered; after entry, recording is authorized; a reused code is rejected.

Test Case Derivation: **FR-HA8** requires a one-time confirmation code sent to the patient's trusted device to be entered into the SLP interface before authorizing any audio recordings.

How test will be performed: Use a mock trusted device to receive the code. Attempt recording without the code and confirm it is blocked. Enter the code and confirm recording is allowed. Reuse the same code and confirm it is rejected.

4.1.3 Speech Analysis

1. ST-F11 Identify listed substitution with position

   Control: Automatic

   Initial State: A patient session is active; target item is selected; the predetermined speech-error list is available; an attempt has been made.

   Input: Audio recording/attempt producing a speech error.

   Output: The system identifies the error from the predetermined list and reports the affected phoneme, the produced phoneme, and the word position.

   Test Case Derivation: **FR-SA1** requires the system to identify speech-errors from a predetermined list and report the expected phoneme, produced phoneme, and word position.

   How test will be performed: Inject the audio through the microphone. Inspect the result and UI to verify the expected phoneme, produced phoneme, and word position fields.

2. ST-F12 Acceptable variant is not a substitution

   Control: Automatic

   Initial State: A patient session is active; A test target item is configured with an acceptable phoneme variant at a given position; audio matching that variant is available.

Input: Audio recording/attempt whose produced phoneme matches the configured acceptable variant.

Output: The system does not identify a substitution and does not report an error for that position.

Test Case Derivation: **FR-SA2** requires the system not to identify a substitution when the produced phoneme matches any acceptable variant defined for the target item.

How test will be performed: Configure the acceptable variant in the test data. Inject the matching audio and inspect the result to confirm no substitution is flagged.

3. ST-F13 Confidence reporting and unclassifiable result

   Control: Automatic

   Initial State: A patient session is active; A confidence threshold is configured.

   Input: Analyze a high-confidence sample/recording; analyze a low-confidence sample/recording.

   Output: The high-confidence result reports a confidence value and is classified; the low-confidence result is classified as "unclassifiable"; each phoneme deduction includes a confidence percentage.

   Test Case Derivation: **FR-SA3** requires a confidence value with each result and an "unclassifiable" classification below the confidence threshold. **FR-HA3** requires the speech engine to provide a confidence percentage for each phoneme deduction.

   How test will be performed: Inject both samples. Inspect the result fields for per-phoneme confidence and classification status, and verify the threshold behavior.

4.1.4 Results and Data Visualization

1. ST-F14 Expected vs recognized phonemes, errors, position, and IPA

   Control: Manual

Initial State: An attempt with an identified error at the beginning of the word is available.

Input: Open the attempt result view/visualization.

Output: Expected phonemes and recognized phonemes are shown side by side; the word position visualization indicates "beginning"; phonemes are displayed as IPA symbols.

Test Case Derivation: **FR-RDV1** requires expected and recognized phonemes to be presented. **FR-RDV2** requires substitution errors to indicate which expected phoneme was replaced by which produced phoneme. **FR-RDV3** requires a visualization of word position. **FR-RDV9** requires phonemes to be presented as IPA symbols.

How test will be performed: Open the result view and compare the displayed expected and recognized sequences. Verify the substitution label, the position visualization, and the IPA rendering.

2. ST-F15 Session summary, heat map, and live update

   Control: Manual

   Initial State: A patient session is active with several correct/incorrect and independent/cued attempts; heat-map data is available.

   Input: Complete attempts; open the session visualization; complete one more attempt.

   Output: The session visualization reports correct/incorrect counts and independent/cued counts; the heat map indicates the closeness of the produced phoneme to the expected phoneme; the visualization updates as the session progresses and completes.

   Test Case Derivation: **FR-RDV4** requires a session visualization of correct/incorrect and independent/cued attempts. **FR-RDV5** requires a heat-map visualization of produced versus expected phoneme closeness. **FR-RDV6** requires visualizations to update as the session is completed.

   How test will be performed: Perform attempts through the UI. Capture the visualization before and after the final attempt and verify the counts, the heat-map presence, and the update behavior.

3. ST-F16 Assistive disclaimer and non-discouraging error display

   Control: Manual

   Initial State: Session visualizations are available.

   Input: View all deduction and analytics screens; inspect the disclaimer and the error presentation.

   Output: All visualizations are identified as assistive and not diagnostic; a disclaimer or acknowledgement precedes deductions and analytics; error information is not presented in a way that discourages the patient.

   Test Case Derivation: **FR-RDV7** requires all visualizations to be identified as assistive and not diagnostic. **FR-RDV8** requires the system not to present error information in a way that discourages the patient. **FR-HA7** requires all deductions and analytics to be prefaced with an acknowledgement that they may be incorrect and may only be accepted under the discretion of an SLP.

   How test will be performed: Review each visualization screen and verify the disclaimer text appears before results. Conduct an SLP review of the error wording and visuals to confirm they are not discouraging.

4.1.5 Progress and Data Management

1. ST-F17 Patient record, session record, and patient association

   Control: Manual

   Initial State: Two patients exist; a session is run for one of them.

   Input: Create and view a patient record; run a session with targets and SLP inputs; view and export the session history.

   Output: The patient's identifying information and session history are maintained; the session record includes the date, SLP, target phonemes/words, session results, and SLP inputs; the session results are associated only with the session patient.

   Test Case Derivation: **FR-DMR1** requires a record of identifying information and session history for each patient. **FR-DMR6** requires

each session to record the date, SLP, target phonemes and words, session results, and SLP inputs. **FR-DMR7** requires session results to be associated only with the session patient.

How test will be performed: Create both patients and run a session for one. Inspect the database and export to verify all required session fields are present and no data from the other patient is linked.

2. ST-F18 Support level, correction, override, and cue/repetition flag

   Control: Manual

   Initial State: A session is active with attempts classified by the system.

   Input: The SLP inputs a support level; corrects the system output; marks an attempt correct or incorrect regardless of system output; records whether a correct item required cues or repetition.

   Output: The support level is stored with the correct attempt; the correction and override are stored; the cue/repetition flag is recorded for the correct practiced item.

   Test Case Derivation: **FR-DMR2** requires the support level to be recorded with the attempt it applies to. **FR-DMR2.1** allows the SLP to input the level of support per attempt. **FR-DMR3** allows the SLP to correct system output. **FR-DMR3.1** allows the SLP to mark an attempt correct or incorrect regardless of recognized output. **FR-DMR11** requires recording whether a correct practiced item required additional cues or repetition.

   How test will be performed: Perform each SLP action through the UI and query the attempt records to verify the stored support level, correction, override, and cue/repetition flag.

3. ST-F19 Audit log creation, retrieval, and copying

   Control: Manual

   Initial State: An authorized SLP or admin account exists; a patient record exists.

   Input: View, create, modify, export, and delete a patient record; retrieve the audit log; copy the audit log.

Output: Each event creates an audit entry containing the information type, date and time, user identity, and patient identity; an authorized user can retrieve and copy the audit log.

Test Case Derivation: **FR-DMR4** requires an audit entry for every view, creation, modification, export, or deletion of a patient record, including the information type, date and time, user identity, and patient identity. **FR-DMR5** allows authorized users to retrieve and copy the audit log. **FR-HA9** requires an audit trail detailing the user, action, and time of all data accesses and manipulations.

How test will be performed: Perform each record action and inspect the audit log. Verify the required fields and timestamps. Retrieve and copy the log as an authorized user.

4. ST-F20 Longitudinal performance tracking and comparison

   Control: Manual

   Initial State: A patient has sessions over multiple dates with one target phoneme improving and another showing persistent errors.

   Input: Open the progress view; compare sessions.

   Output: The system tracks performance on target phonemes over time; the SLP can compare performance between sessions; the system identifies improving target phonemes and persistent errors.

   Test Case Derivation: **FR-DMR8** requires tracking each patient's performance on target phonemes over time. **FR-DMR9** allows the SLP to compare performance between sessions. **FR-DMR10** requires tracking and identifying target phonemes showing improvement and those with persistent errors.

   How test will be performed: Seed session data for the patient. Open the progress and comparison views and verify the trends, comparison outputs, and improving/persistent classifications.

5. ST-F21 Session-end backup and failure alert

   Control: Automatic

Initial State: A patient session with progress data is active; backup storage is available; a backup failure can be injected.

Input: End the session normally; end a second session with backup failure injected.

Output: All patient progress data and analytics are backed up at the end of the session; if the backup fails, the user is alerted.

Test Case Derivation: **FR-HA2** requires the system to back up all patient progress data and analytics at the end of each session and to alert the user if the backup fails.

How test will be performed: End a normal session and verify the backup. Inject a backup failure, end another session, and observe the alert to confirm the failure is surfaced.

4.1.6 Practice Activities

1. ST-F22 Practice items, quantity, and standardized set

   Control: Automatic

   Initial State: The practice item catalog is loaded.

   Input: Query the catalog for a target phoneme in a given word position; inspect item metadata and counts.

   Output: Each item is associated with its target phoneme(s) and word position; at least three items exist per target phoneme per word position; items are appropriate for ages 4–6 and drawn from a standardized word set.

   Test Case Derivation: **FR-PAR1** requires practice items associated with the target phoneme(s) and word position. **FR-PAR2** requires at least three practice items for each target phoneme in each word position. **FR-PAR4** requires items to be appropriate for the supported age range and drawn from a standardized set of words.

   How test will be performed: Query the catalog or API and count items per target and position. Review the metadata and source labels for standardized-set and age-appropriateness.

2. ST-F23 Selection by target/position and SLP practice controls

   Control: Manual

   Initial State: The practice catalog is loaded; a session is active.

   Input: Practice items are selected by target phoneme and word position; then the next, repeat, skip, and end practice controls are used.

   Output: Selection filters items correctly; the SLP can move to the next item, repeat an item, skip an item, or end practice at any time.

   Test Case Derivation: **FR-PAR3** requires the SLP to select practice items by target phoneme and word position. **FR-PAR6** allows the SLP to move to the next item, repeat an item, skip an item, or end practice at any time.

   How test will be performed: Use the filter controls and verify the displayed items. Execute each practice control and verify the session state.

3. ST-F24 Image-cue prompt, appropriateness, and engagement

   Control: Manual

   Initial State: A practice item is selected; prompt assets are available.

   Input: Display the prompt; have the child respond without the SLP modeling the item; inspect engagement elements.

   Output: The prompt is an image cue rather than a written word; the prompt is recognizable and appropriate; it elicits production without the SLP saying the item first; engagement elements are present.

   Test Case Derivation: **FR-PAR5** requires a prompt that elicits the child to produce the item without the SLP saying it first. **FR-PAR5.1** requires prompts in the form of image cues rather than written words. **FR-PAR5.2** requires prompts to be recognizable and appropriate. **FR-PAR7** requires elements that keep patients engaged during practice.

   How test will be performed: Review the prompt assets and conduct an SLP review for appropriateness. Observe a child responding without SLP modeling and verify the engagement elements are present.

4.1.7 Reliability and Offline Behavior

1. ST-F25 WiFi instability alert and offline audio recording

   Control: Automatic

   Initial State: An online session is active with stable WiFi; offline mode is available.

   Input: Degrade the WiFi to unstable; attempt recording; restore WiFi.

   Output: The system alerts the user that WiFi is unstable; the offline version records patient audio while offline; the recorded audio is available for processing when WiFi recovers.

   Test Case Derivation: **FR-HA4** requires the system to alert the user when WiFi becomes unstable and to provide an offline version that records patient audio for when WiFi recovers.

   How test will be performed: Throttle or disconnect the network and observe the alert. Record audio offline and restore the network. Verify the queued audio is uploaded or processed.

2. ST-F26 Online-to-offline transfer cannot be blocked

   Control: Automatic

   Initial State: The system is online; a session is active; offline mode is available.

   Input: Force a network drop during operation; inject an interface malfunction during the online-to-offline transfer.

   Output: The system transfers from online to offline at all times; the transfer is not blocked by network or interface malfunction; the session continues with limited functionality and fails gracefully.

   Test Case Derivation: **FR-HA5** requires the system to transfer from online to offline mode at all times, with the transfer unable to be blocked by network or interface malfunctions.

   How test will be performed: Toggle the network and mock an interface failure. Observe the mode transition and verify the session remains usable for limited functions without freezing.

4.2 Tests for Nonfunctional Requirements

4.2.1 Accuracy and responsiveness

Use a consistent evaluation set so measurements can be compared across model changes, as required by the Development Plan.

Measure phoneme detection accuracy

1. **ST-N01**

   Type: Dynamic, Automatic

   Initial State: A set of labelled children's speech clips from TalkBank (e.g. PhonBank), with reference phoneme transcriptions, is loaded. Clips used for tuning are excluded.

   Input/Condition: Run every clip through the recognizer and compare the output phonemes against the labelled reference phonemes.

   Output/Result: Phoneme identification accuracy is at least 87% (NFR-ACCU1; FR-SA1, FR-SA2, FR-SA3). Report accuracy overall and per target phoneme.

   How test will be performed: Align recognized and reference phonemes for each clip. Accuracy is the number of correctly identified phonemes divided by the total number of reference phonemes. Rerun the same clip set after each model change.

Measure processing time from audio capture to phoneme estimation

1. **ST-N02**

   Type: Dynamic, Automatic measurement

   Initial State: The device and software are configured to record speech and process it.

   Input/Condition: Input audio file for a particular word.

   Output/Result: For each case, phonemic estimation results are visible in less than 10 seconds (NFR-PER1). Record duration for each processing completion and categorize by the specific phrase. Report all

observations plus processing time divided by audio duration, median, and maximum.

How test will be performed: Measure from completed audio capture/submission to rendered result, including transfer and rendering, with at least 30 runs per representative condition as a planned sampling choice. Log hardware, network, clip duration and model size. Resolve whether capture time belongs in the requirement's timing boundary before acceptance. Basis: NFR-PER1 sets a strict less than 10 seconds criterion; Development Plan Section 10.5 adds real-time factor and end to end latency.

4.2.2 Usability and patient protection

Use SLP task observations for navigation and efficiency. Assess the child-facing boundary separately from clinician access.

Learn the workflow within one hour

1. **ST-N03**

   Type: Dynamic, Manual

   Initial State: An SLP unfamiliar with the application has standard onboarding material and synthetic data.

   Input/Condition: Within a one hour session, ask the SLP to configure practice, record audio and analyze it, override a result, inspect charts and observe patient history.

   Output/Result: The SLP completes the core navigation tasks independently within one hour of use (NFR-USA1). Record task times, assistance, failures and comments.

   How test will be performed: The UI/UX Lead observes without guiding the final task run. They record mistakes, time to complete each task, and any questions or comments. The SLP is asked to provide feedback on the experience.

1. **ST-N04**

Type: Dynamic usability and recovery checks

Initial State: A test SLP account exists with synthetic critical data (for example, patient records and session recordings).

Input/Condition: As the SLP, attempt to delete each type of critical data item. Where a confirmation prompt appears, test both cancelling and confirming. Where deletion is allowed without a prompt, attempt recovery at several points within 72 hours and again after 72 hours have elapsed.

Output/Result: No critical data item is permanently deleted without either a confirmation step or a recovery option (NFR-USA2). Cancelling a confirmation leaves the data unchanged. Deleted data is fully restored when recovery is requested within 72 hours, with contents identical to the original.

How test will be performed: The Development Lead and an independent reviewer walk through each data type, automated script compares restored contents against the original synthetic data, and check the system clock or retention settings to verify the 72-hour window. Record any data type that can be lost without a safeguard; review is blocked if the critical data list is absent. Basis: NFR-USA2.

Compare workflow time with manual tools

1. **ST-N05**
   Type: Dynamic, Manual comparative measurement

   Initial State: An SLP has learned the application and has equivalent paper or spreadsheet tools.

   Input/Condition: Complete matched session and after-session documentation tasks with identical attempts using software and manual tools.

   Output/Result: Total software task time is lower than manual task time, with equivalent completed records (NFR-USA3). Report paired times, corrections and assistance; a faster incomplete record does not pass.

   How test will be performed: The UI/UX Lead checks where multiple participants are available and records session and after-session times

separately and together. Report sample size and learning effects. Basis: NFR-USA3 requires less time than equivalent manual tools, including work outside the session.

Restrict the child-facing experience

1. **ST-N06**

   Type: Dynamic, Mixed manual and automatic

   Initial State: Child practice mode is active; clinician functions require the configured access boundary.

   Input/Condition: Try visible controls, back navigation, direct clinician URLs and direct API requests from the child context.

   Output/Result: The child can use the gamified experience but cannot access patient records, analytics administration or clinician controls beyond that experience (NFR-USA4). Direct navigation and API calls do not bypass restrictions.

   How test will be performed: The Systems Architect/Development Lead tests each route with the UI/UX Lead and captures denied requests and screenshots. Verify returning to clinician mode requires the configured authorization. Basis: NFR-USA4 limits the child's access to the practice experience.

4.2.3 Maintainability and portability

Checks supported hardware and software environments. Check deployment, patching, and extension.

1. **ST-N07**

   Type: Dynamic performance check

   Initial State: A clean build environment and a prepared patch exist, and the approved build and deployment pipeline is available.

   Input/Condition: Apply a representative patch and run the pipeline from build start to a completed deployment, timing each stage.

Output/Result: Building and deploying the patch completes successfully in less than 30 minutes (NFR-MAI3).

How test will be performed: The Development Lead and an independent reviewer time the pipeline run from pipeline logs and repeat it three times. Record durations and any failures; review is blocked if the pipeline is unavailable. Basis: NFR-MAI3.

Verify platform and browser compatibility

1. **ST-N08**

   Type: Dynamic, Manual

   Initial State: A versioned compatibility matrix is recorded with physical devices for microphone checks.

   Input/Condition: Exercise Windows with Chrome, Firefox and Edge; macOS with Safari, Chrome and Firefox; iPad/iOS with Safari; and Android tablet with Chrome. Include desktop, laptop and tablet form factors.

   Output/Result: Setup, microphone permission and capture, analysis, SLP controls, IPA, charts and history operate without blocking layout or functional defects in every tested combination (NFR-POR1, NFR-POR2, NFR-POR3; FR-RDV9).

   How test will be performed: Repeat a compact ST-F01/F03/F09/F10/F12 workflow in each matrix entry; inspect orientation and touch controls on tablets; test microphone functionality. Add any other claimed supported combination to the matrix before declaring support. Basis: The SRS lists four operating systems, four browsers and three device types; the matrix is a representative test plan, not proof of every OS-browser combination.

4.2.4 Reliability, security and release review

These checks address operational hazards, including data loss, and unauthorized access.

Verify operating periods and maintenance scheduling

1. **ST-N09**

   Type: Mixed Dynamic and Static

   Initial State: Deployment operating timezone, full and minimal service definitions, load and maintenance policy are approved.

   Input/Condition: Exercise login, storage, retrieval and analysis around 06:00 and 00:00; run an accelerated schedule check and inspect the maintenance calendar.

   Output/Result: The approved full service level operates from 06:00 to 00:00 and the approved minimum remains available from 00:00 to 06:00. Routine maintenance is scheduled in minimal hours at the approved interpretation of the one to two month interval (NFR-REL2, NFR-REL3, NFR-REL4). Check server latency and ensure it is under 10 seconds for this period (NFR-REL1).

   How test will be performed: Use a controlled clock and scheduler inspection, then planned observation under the approved workload.

Protect data and secure communications

1. **ST-N10**

   Type: Static inspection and Dynamic security checks

   Initial State: The approved PHI inventory and encryption baseline identify every component and storage location.

   Input/Condition: Create and transfer synthetic PHI; inspect browser-to-API, service-to-service and database traffic, persistent stores, logs, temporary files and backups.

   Output/Result: PHI is encrypted from collection through its approved storage lifecycle; intercomponent traffic uses the approved secure protocol (NFR-SAP1, NFR-SAP2; NFR-LAL1). No unintended plaintext PHI appears in inspected storage or traffic. Approved key access and certificate validation controls are active.

How test will be performed: The Systems Architect/Development Lead and an independent team reviewer inspect configuration, packet traces and synthetic storage samples. Test denial of an untrusted certificate and an insecure endpoint. Record coverage and gaps; review is blocked if the baseline is absent. Basis: NFR-SAP1 and NFR-SAP2 require encryption; exact cryptographic acceptance comes from an approved security baseline, not this plan.

4.2.5 Legal and Licensing

These checks assess the system against the approved PHIPA checklist and verify the project's open-source licensing requirements.

1. **ST-N11**

   Type: Static inspection and Dynamic security checks

   Initial State: The approved PHIPA checklist and deployment region list exist.

   Input/Condition: Process synthetic patient records through storage, transfer and access paths; attempt access with unauthorized roles; review the repository license and dependency manifest.

   Output/Result: Synthetic PHI is stored, transferred and accessed only as the PHIPA checklist permits, and stays in approved residency locations (NFR-LAL1). Unauthorized access is denied. The repository includes the GNU GPL text and all dependencies are GPL-compatible (NFR-LAL2).

   How test will be performed: The Development Lead and an independent reviewer compare configuration, storage locations and access logs against the checklist, and run a dependency license scan and manually review flagged or unknown licenses for compatibility with the project's chosen GPL version. Record findings; review is blocked if the checklist is absent. Basis: NFR-LAL1 and NFR-LAL2; legal sign-off is outside this plan.

4.3 Traceability Between Test Cases and Requirements

The following matrix maps system test cases to the requirements they support. Inclusion in the matrix indicates planned test coverage, not successful execution.

Table 2: Traceability Between System Tests and Requirements

Test Case	Requirement(s)
ST-F01	FR-AC1
ST-F02	FR-AC2
ST-F03	FR-AC3
ST-F04	FR-HA1
ST-F05	FR-SM1, FR-SA1
ST-F06	FR-SM2
ST-F07	FR-SM3
ST-F08	FR-SM4
ST-F09	FR-HA6
ST-F10	FR-HA8
ST-F11	FR-SA1
ST-F12	FR-SA2
ST-F13	FR-SA3, FR-HA3
ST-F14	FR-RDV1, FR-RDV2, FR-RDV3, FR-RDV9
ST-F15	FR-RDV4, FR-RDV5, FR-RDV6
ST-F16	FR-RDV7, FR-RDV8, FR-HA7
ST-F17	FR-DMR1, FR-DMR6, FR-DMR7
ST-F18	FR-DMR2, FR-DMR2.1, FR-DMR3, FR-DMR3.1, FR-DMR11
ST-F19	FR-DMR4, FR-DMR5, FR-HA9
ST-F20	FR-DMR8, FR-DMR9, FR-DMR10
ST-F21	FR-HA2

Continued on next page

Table continued

Test Case	Requirement(s)
ST-F22	FR-PAR1, FR-PAR2, FR-PAR4
ST-F23	FR-PAR3, FR-PAR6
ST-F24	FR-PAR5, FR-PAR5.1, FR-PAR5.2, FR-PAR7
ST-F25	FR-HA4
ST-F26	FR-HA5
ST-N01	NFR-ACCU1, FR-SA1, FR-SA2, FR-SA3
ST-N02	NFR-PER1
ST-N03	NFR-USA1
ST-N04	NFR-USA2
ST-N05	NFR-USA3
ST-N06	NFR-USA4
ST-N07	NFR-MAI3
ST-N08	NFR-POR1, NFR-POR2, NFR-POR3, FR-RDV9
ST-N09	NFR-REL1, NFR-REL2, NFR-REL3, NFR-REL4
ST-N10	NFR-SAP1, NFR-SAP2, NFR-LAL1
ST-N11	NFR-LAL1, NFR-LAL2

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here,

you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

References

Author Author. System requirements specification. https://github.com/..., 2019.

6 Appendix

6.1 Symbolic Parameters

Not applicable at the moment.

7 Appendix — Generative AI Use Disclosure

7.1 AI Tools Used

Table 3: List of AI Tools Used

Tool Number	Name	Model
1	ChatGPT	GPT 5 Thinking Mini
2	Claude	Sonnet 5
3	ChatGPT	GPT 6 Astra

7.2 How and Where Used

In general, generative AI was used to help with LaTeX formatting. More specific uses may be noted in the table below.

Table 4: Usage Details for All AI Tools Used

Tool Number	Location	Details
1	(3) Plan	Explained what the difference between each section was because some sections were slightly confusing (3.4, 3.7).
1	(3) Plan	Gave the LLM the write-up for 3.4, 3.5, 3.6, and asked if there were any key strategies missed.
2	(8) Reflection	Gave the AI a rough draft and bullet points for the reflection answers. Asked it to generate it into a nice paragraph, and edited the generated paragraph manually.
3	(4) System Tests	Used the AI for ideation and brainstorming of system tests given the SRS requirements. This was done to cover a broad scope of testing in addition to the manually brainstormed tests. All generated tests were meticulously edited to match all characteristics desired of PronouncedFish.

7.3 Verification of AI-Generated Content

Verification of AI content involved a human reviewer for any and all generated content. For LaTeX formatting, as long as it compiled it was fine. For all uses in parts (3) and (4), all generated content was read and taken as brainstorming. Part (3) used it to uncover missing strategies in the current writing. Each suggestion was weighed on if it was adding enough value to the VnV to offset the added complexity. Having more validation is directly correlated with putting in more testing effort. However, at some point putting in more effort becomes infeasible due to timing constraints, so only the most

important strategies were used. For part (4), the goal was to get a diverse pool of tests that might have been missed during the team's ideation session. No test was directly transferred to the VnV document without proper verification. Each one was read, and ones that fit with the group's vision and SRS were used as a template. This template had all inputs, outputs, and procedures revised with more precise human wording instead of the generic LLM generated one.

8 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
   Many of our requirements were specific and measurable, which made it straightforward to derive test cases for both our functional and non-functional requirements. Our usability requirements in particular came with concrete, verifiable targets. For example, one requirement states that a speech-language pathologist (SLP) should be able to fully learn the system within an hour. Because each requirement had a clear pass/fail criterion, we could map requirements to tests with very little ambiguity or debate. This also gave us confidence that our test plan covers what the system is actually supposed to do, and it will make it easier to show traceability from requirements to tests later in the project.

2. What pain points did you experience during this deliverable, and how did you resolve them?
   Writing a test plan requires at least a basic understanding of the whole system in order to anticipate which tests will be needed, and it was difficult to hold the entire system in our heads at once. This made it easy to overlook interactions between components or to focus too heavily on one area. To address this, we drew diagrams of the system

and walked through them together. The diagrams gave us a shared view of how the pieces fit together and helped us identify specific points of contention, meaning the places where components interact or where failures are most likely, so we could direct our testing effort there.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.
   To successfully complete verification and validation, our team will need to learn **pytest**, **usability testing methods**, and **CI/CD automation**.

4. **pytest** will be our main tool for automating functional tests, so we will need to learn how to structure test suites, write clear test cases, and use features such as fixtures and parametrization.

5. **Usability testing methods** are needed to validate requirements that depend on real users, such as the SLP learning the system within an hour. We will need to understand how to design test sessions, define tasks, and collect meaningful results.

6. **CI/CD automation** will let our tests run automatically on every change. We already have a CI/CD pipeline, but we will need to expand it. To do that, we first need to understand how the current pipeline works, including its stages, configuration, and triggers, so that we can extend it to run our tests reliably without breaking existing behaviour.

7. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?
   To successfully complete verification and validation, our team will need to learn **pytest**, **usability testing methods**, and **CI/CD automation**.

8. **pytest** will be our main tool for automating functional tests, so we will need to learn how to structure test suites, write clear test cases, and use features such as fixtures and parametrization.

9. **Usability testing methods** are needed to validate requirements that depend on real users, such as the SLP learning the system within an hour. We will need to understand how to design test sessions, define tasks, and collect meaningful results.

10. **CI/CD automation** will let our tests run automatically on every change. We already have a CI/CD pipeline, but we will need to expand it. To do that, we first need to understand how the current pipeline works, including its stages, configuration, and triggers, so that we can extend it to run our tests reliably without breaking existing behaviour.